

SIMATS ENGINEERING ASSESSMENT TOOL 3

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN

COURSE CODE: CSA11

COURSE OUTCOME COVERED

CO1: *Apply object-oriented concepts and software development life cycle models to design structured and modular software systems. (BL2, BL3)*

Assessment Tool Weightage: 30%

Dr Ramamoorthy

QUESTIONS: 6

Total Marks:30

Annexure A

SHORT ANSWER TEST

Code of Conduct:

*I E.Sakthi priya (192520041) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

E. Sakthi priya

Annexure A

Short Answer Test

1. Suppose a software system is developed for a **Smart Banking System** where customers can open accounts, transfer funds, and pay utility bills.

(a) Explain the following object-oriented concepts:

- Class
- Object
- Message Passing
- Encapsulation
- Polymorphism

(b) Describe the following object relationships:

- Association
- Aggregation
- Composition

(c) Explain how these concepts improve software reusability and maintainability in a banking system.

(5 Marks)

2. Consider developing a **Smart Healthcare Management System**.

(a) Explain the following object-oriented design principles:

- Abstraction
- Encapsulation
- Modularity

(b) Describe the following SOLID principles:

- Liskov Substitution Principle (LSP)
- Interface Segregation Principle (ISP)

(c) Explain how these principles improve software quality and flexibility.

(5 Marks)

3. Suppose a software company is developing an **Online Food Delivery System**.

(a) Explain the phases of the Software Development Life Cycle (SDLC):

- Planning
- Requirement Analysis
- Design
- Development
- Testing
- Maintenance

(b) Describe the purpose of each SDLC phase.

(c) Explain how SDLC contributes to delivering reliable software.

(5 Marks)

4. Consider a project for developing a **Smart Agriculture Monitoring System**, where customer requirements change regularly.

(a) Explain the following SDLC models:

- Prototype Model
- Spiral Model
- Agile Model
- DevOps Model

(b) Compare the Prototype Model and Agile Model.

(c) Recommend the most suitable SDLC model for this project with justification.

(5 Marks)

5. Suppose a software analyst is designing a **Smart Transport Management System**.

(a) Explain the importance of UML in object-oriented software development.

(b) Describe the following UML diagrams:

- State Diagram
- Component Diagram
- Deployment Diagram
- Package Diagram

(c) Explain how UML models improve communication among stakeholders.

(5 Marks)

6. Consider developing a **Smart Inventory Management System**.

- (a)** Explain the concept of modular software design.
- (b)** Compare the procedural programming approach with the object-oriented programming approach.
- (c)** Explain how modularity improves software scalability, maintainability, and reusability.

(5 Marks)

RUBRICS FOR CALCULATION

SHORT ANSWER TEST

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	20%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps

1. Smart Banking System

A Smart Banking System allows customers to open accounts, transfer funds, and pay utility bills.

(a) Object-Oriented Concepts

1. Class

A class is a blueprint used to create objects. It defines the data and functions of an object.

Example: Customer, Account, Transaction, and Bank are classes.

2. Object

An object is an instance of a class. It represents a real-world entity with its own data and behavior.

Example: A customer named Rahul with Account Number **1001** is an object of the **Customer** class.

3. Message Passing

Message passing is the communication between objects by calling methods.

Example: The **Customer** object sends a request to the **Account** object to transfer money.

4. Encapsulation

Encapsulation means combining data and methods into a single unit and protecting the data from unauthorized access.

Example: The account balance is private and can only be accessed using methods like **deposit()** and **withdraw()**.

5. Polymorphism

Polymorphism allows the same method to perform different actions depending on the object.

Example: The method **payBill()** can be used to pay electricity, water, or internet bills with different implementations.

(b) Object Relationships

1. Association

Association is a relationship where two objects work together but can exist independently.

Example: A **Customer** is associated with an **Account**.

2. Aggregation

Aggregation is a "has-a" relationship where one object contains another, but both can exist separately.

Example: A **Bank** has many **Customers**. Even if a customer closes the account, the bank still exists.

3. Composition

Composition is a strong "part-of" relationship where one object cannot exist without the other.

Example: A **Transaction** belongs to an **Account**. If the account is deleted, its transaction records are also removed.

(c) How These Concepts Improve Reusability and Maintainability

- Classes allow code to be reused for creating multiple customer and account objects.
- Encapsulation protects sensitive information like account balance and passwords.
- Polymorphism makes it easy to add new payment methods without changing existing code.
- Association, aggregation, and composition organize relationships between objects, making the system easier to understand.
- These concepts reduce code duplication, improve security, simplify maintenance, and make the banking system scalable.

Conclusion:

Object-oriented concepts help build a Smart Banking System that is secure, reusable, maintainable, and easy to upgrade with new banking features.

2. Smart Healthcare Management System

A Smart Healthcare Management System helps manage patients, doctors, appointments, medical records, and billing efficiently.

(a) Object-Oriented Design Principles

1. Abstraction

Abstraction means showing only the essential features of an object while hiding unnecessary details.

Example: A patient can book an appointment without knowing how the appointment is stored in the database.

2. Encapsulation

Encapsulation means combining data and methods into one unit and protecting the data from unauthorized access.

Example: A patient's medical records are private and can only be accessed through authorized methods.

3. Modularity

Modularity means dividing the system into smaller independent modules.

Example: The healthcare system can have separate modules for Patient Management, Doctor Management, Appointments, Billing, and Pharmacy.

(b) SOLID Principles

1. Liskov Substitution Principle (LSP)

LSP states that a subclass should be able to replace its parent class without changing the program's behavior.

Example: If **Doctor** is a parent class, then **Cardiologist** and **Dentist** can be used wherever a **Doctor** object is expected.

2. Interface Segregation Principle (ISP)

ISP states that a class should not be forced to implement methods it does not need. Instead, use small, specific interfaces.

Example: A **Doctor** interface contains methods related to diagnosis, while a **Receptionist** interface contains methods related to appointment scheduling.

(c) How These Principles Improve Software Quality and Flexibility

- Abstraction hides unnecessary details and makes the system easier to use.
- Encapsulation protects sensitive patient information and improves security.
- Modularity allows each module to be developed, tested, and maintained independently.
- LSP enables new types of doctors or staff to be added without affecting existing code.
- ISP reduces unnecessary code and makes the system easier to maintain and extend.

Conclusion:

These object-oriented and SOLID principles improve software quality by making the Smart Healthcare Management System secure, flexible, reusable, and easy to maintain.

3. Online Food Delivery System

An Online Food Delivery System allows customers to order food online, make payments, and track deliveries.

(a) Phases of the Software Development Life Cycle (SDLC)

1. Planning

In this phase, the project goals, budget, timeline, and resources are decided.

2. Requirement Analysis

The needs of customers and businesses are collected and analyzed to understand what the system should do.

3. Design

The system structure, database, user interface, and software architecture are designed.

4. Development

Programmers write the code and develop the features of the online food delivery system.

5. Testing

The software is tested to identify and fix errors before it is released.

6. Maintenance

After deployment, the software is updated, bugs are fixed, and new features are added.

(b) Purpose of Each SDLC Phase

- **Planning:** Defines project objectives, budget, and schedule.
- **Requirement Analysis:** Identifies user needs and system requirements.
- **Design:** Creates the blueprint for software development.
- **Development:** Builds the software according to the design.
- **Testing:** Ensures the software works correctly and is free from defects.
- **Maintenance:** Improves the software by fixing issues and providing updates after release.

(c) How SDLC Contributes to Delivering Reliable Software

- Ensures all customer requirements are clearly understood.
- Helps detect and fix errors before the software is released.
- Improves software quality through proper planning and testing.
- Makes the software easier to maintain and update.
- Delivers a reliable, secure, and user-friendly Online Food Delivery System.

Conclusion:

The SDLC provides a structured approach to software development, ensuring that the Online Food Delivery System is reliable, efficient, and meets user requirements.

4. Smart Agriculture Monitoring System

A Smart Agriculture Monitoring System helps farmers monitor crops, soil moisture, weather conditions, and irrigation. Since customer requirements change frequently, the system should be flexible.

(a) SDLC Models

1. Prototype Model

A prototype (sample version) of the software is developed first. Users provide feedback, and the system is improved before the final product is built.

2. Spiral Model

The Spiral Model combines development and risk analysis. The project is developed in repeated cycles, with risks identified and reduced in each cycle.

3. Agile Model

The Agile Model develops software in small parts called iterations. Customer feedback is collected regularly, and changes are made quickly.

4. DevOps Model

The DevOps Model combines software development and operations. It focuses on continuous development, testing, deployment, and maintenance for faster software delivery.

(b) Comparison of Prototype Model and Agile Model

Prototype Model	Agile Model
Builds a sample version first.	Develops the software in small iterations.
Mainly used to understand user requirements.	Focuses on continuous development and frequent customer feedback.
Customer feedback is taken after the prototype.	Customer feedback is taken throughout the project.
Suitable when requirements are unclear.	Suitable when requirements change frequently.

(c) Most Suitable SDLC Model with Justification

The **Agile Model** is the most suitable for a Smart Agriculture Monitoring System because customer requirements change regularly.

Justification:

- Supports frequent requirement changes.
- Delivers software in small, working versions.
- Encourages continuous customer feedback.
- Improves software quality through regular testing.
- Allows new features to be added quickly without affecting the entire system.

Conclusion:

The Agile Model is the best choice because it is flexible, customer-focused, and supports continuous improvement, making it ideal for projects with changing requirements.

5. Smart Transport Management System

A Smart Transport Management System helps manage buses, trains, routes, passengers, ticket booking, and vehicle tracking efficiently.

(a) Importance of UML in Object-Oriented Software Development

UML (Unified Modeling Language) is a standard modeling language used to design and visualize object-oriented software systems before coding.

Importance of UML:

- Helps developers understand the system clearly.
- Provides a visual representation of the software design.
- Improves communication among developers, analysts, and clients.
- Identifies design problems before implementation.
- Makes software development and maintenance easier.

(b) UML Diagrams

1. State Diagram

A State Diagram shows the different states of an object and how it changes from one state to another based on events.

Example: A bus ticket can move through the states **Booked** → **Confirmed** → **Cancelled**.

2. Component Diagram

A Component Diagram shows the different software components and how they interact with each other.

Example: The Smart Transport System contains components such as **Booking Module**, **Payment Module**, **Route Management Module**, and **Notification Module**.

3. Deployment Diagram

A Deployment Diagram shows how software is deployed on hardware devices and servers.

Example: The transport application is deployed on **User Mobile**, **Application Server**, and **Database Server**.

4. Package Diagram

A Package Diagram organizes related classes into packages to improve system structure.

Example: Packages include **Passenger**, **Vehicle**, **Booking**, **Payment**, and **Admin**.

(c) How UML Models Improve Communication Among Stakeholders

- UML diagrams provide a clear visual representation of the system.
- Developers, analysts, testers, and clients can easily understand the software design.
- They reduce misunderstandings and improve communication.
- They help identify design errors before coding begins.
- They make the software easier to develop, maintain, and update.

Conclusion:

UML models improve planning, communication, and documentation, helping stakeholders work together to develop a reliable and well-structured Smart Transport Management System.

6. Smart Inventory Management System

A Smart Inventory Management System helps businesses manage products, stock levels, suppliers, orders, and sales efficiently.

(a) Concept of Modular Software Design

Modular software design is the process of dividing a software system into small, independent modules. Each module performs a specific function and can be developed, tested, and maintained separately.

Example: A Smart Inventory Management System can have separate modules for:

- Product Management
- Stock Management
- Supplier Management
- Order Management
- Report Generation

This makes the software easier to understand and manage.

(b) Comparison of Procedural Programming and Object-Oriented Programming

Procedural Programming	Object-Oriented Programming (OOP)
Focuses on functions and procedures.	Focuses on objects and classes.
Data and functions are separate.	Data and methods are combined into objects.
Less secure because data can be accessed directly.	More secure through encapsulation.

Procedural Programming	Object-Oriented Programming (OOP)
Code reusability is limited.	Supports code reusability through inheritance and polymorphism.
Suitable for small programs.	Suitable for large and complex software systems.

(c) How Modularity Improves Software Scalability, Maintainability, and Reusability

- **Scalability:** New modules, such as Barcode Scanning or Warehouse Management, can be added without affecting existing modules.
- **Maintainability:** Errors can be fixed in one module without changing the entire system.
- **Reusability:** Existing modules can be reused in other inventory or business applications, reducing development time.
- Independent modules make testing easier.
- The system becomes more organized, flexible, and easier to upgrade.

Conclusion:

Modular software design helps build a Smart Inventory Management System that is scalable, maintainable, reusable, and easy to develop, making it suitable for future business growth.